Assignment 2: Optimised Dense and Sparse Matrix Multiplication

Big Data – Degree in Data Science and Engineering (ULPGC)

Author: Sara Dévora Ortega | **Date:** 26 October 2026

Abstract

In this assignment I study two questions about matrix multiplication. In Part A, I compare a basic dense *I-J-K* implementation in C with two optimised versions, loop reordering (*I-K-J*) and cache blocking, for sizes from $n = 10$ to $n = 1000$. In Part B, I compare sparse multiplication in CSR format (SciPy) with dense multiplication (NumPy) for densities of 1%, 5%, 10% and 20%. In my tests, both optimisations were faster than the baseline, with blocking being the best option for $n \geq 500$ (up to about 5 times faster than the baseline). The sparse version only won at 1% density and $n \geq 500$, and the product became almost fully dense from 5% on. The measurements are noisy (Windows, 5 repetitions), so the results are limited to this machine and setup.

1 Introduction

Matrix multiplication is a very common operation in data science, but the simple triple loop is slow for large matrices. In Assignment 1 I measured the basic *I-J-K* algorithm in three languages. Here I stay with C and ask two questions:

1. **Part A:** how much do loop reordering and cache blocking improve a dense baseline, and does the block size B matter?
2. **Part B:** for which tested sizes and densities is a sparse representation (CSR) faster than a dense multiplication, taking into account that the product can become denser (*fill-in*)?

2 Background

Dense multiplication and cache. The naive algorithm for $C = A \times B$ takes $O(n^3)$ operations. With matrices stored by rows, the *I-J-K* order reads B column by column, which jumps n elements in memory each step, so I expect many cache misses for large n [1]. Loop reordering to *I-K-J* makes the innermost loop read B and C row by row (sequential access). Cache blocking (tiling) splits the matrices into $B \times B$ blocks so that the data being used stays in cache and is reused [2]. I did not measure cache misses, so these explanations are only expectations.

Sparse matrices. In CSR format only the nonzero values are stored, together with their column indices and one pointer per row. Multiplying two sparse matrices is usually done row by row [3]. The product of two sparse matrices can have many more nonzeros than the inputs. For random matrices with density d the expected number of nonzeros in the result is

$$nnz(C) \approx n^2 \left(1 - (1 - d^2)^n\right), \tag{1}$$

because an entry of C is nonzero if at least one of the n products $a_{ik}b_{kj}$ is nonzero.

3 Methodology

- **Environment:** Windows 11, x86-64, Intel Core i7-11800H (8 cores, 16 threads, L1d: 32 KB, L2: 256 KB per core, L3: 24 MB cache), 16 GB RAM. C code compiled with GCC 13.2.0 and `-O2`; Python 3.11.5, NumPy 1.25.2, SciPy 1.11.1, psutil 5.9.5. The C code is sequential. The BLAS thread limit (`OPENBLAS_NUM_THREADS=1`) was not applied in Part B (see limitations).

- **Methods:**
  - *Part A* (my own C code, `double`): baseline *I-J-K*; loop reordering *I-K-J*; cache blocking with $B \in \{16, 32, 64, 128\}$ (blocks over i, k, j, with the *I-K-J* order inside each block).
  - *Part B* (library code): sparse product with SciPy's CSR format (`A.dot(B)`, `scipy.sparse`) against dense `np.dot` (BLAS) on the same matrices [4, 5]. I did not use my own sparse code in the experiments: the file `sparse_mult.c` in the repository is an unfinished CSR kernel and was not run or tested.
- **Inputs:** Part A: dense matrices with random values in $[0, 1)$ from NumPy (`default_rng(42)`), saved as binary files and read by the C program, so all methods use the same input. Sizes $n \in \{10, 50, 100, 200, 300, 400, 500, 600, 800, 1000\}$. Part B: `scipy.sparse.random` with seed 42, uniform random positions, values in $[0, 1)$, $n \in \{100, 500, 1000\}$, density $d \in \{1\%, 5\%, 10\%, 200/o\}$ (density = percentage of nonzero entries, sparsity = percentage of zeros, so $d = 1\%$ means 99% sparsity). A and B have the same density and exactly round$(d\,n^2)$ nonzeros each. The dense matrices for the dense method are the same matrices converted with `toarray()`.
- **Timing:** Only the multiplication is timed (C: `QueryPerformanceCounter`; Python: `time.perf_counter`). Reading the files, allocation and (in Part B) matrix generation and conversion to dense are outside the timer, so **conversion costs are excluded**. In *I-K-J* and blocking, setting C to zero is inside the timer, which is a small extra cost compared with the baseline. Each configuration has 5 repetitions with no separate warm-up, and I report the median and the IQR (Q3−Q1). In Part A the 5 repetitions run in the same process, and all raw times are in the CSV. In Part B only the median and IQR were saved.
- **Memory:** Part A: working set of the C process, sampled with `psutil` while it runs (peak of the samples). For Part B I only recorded the resident memory of the whole Python process after the multiplication, which includes about 50 MiB of imported libraries and memory from earlier runs, so I do not use it to compare methods.
- **Correctness:** The three dense kernels print a checksum $C_{0,0} + C_{n-1,n-1}$. For every size, all methods give the same value, and it matches NumPy's `A @ B` to the 4 printed decimals (script `check_checksums.py`). I did not run extra tests of the C kernels with identity or zero matrices. For Part B, `validate.py` checks that SciPy's sparse product equals `np.dot` on a 100×100 test matrix with 5% density (`np.allclose`, absolute tolerance 10^{-12}). The tolerance is only used there; the checksum comparison is limited by the printed precision.

4 Results

4.1 Part A: dense optimisation

Table 1 shows the median time of each method. The best blocked version for each n is in bold. All raw repetitions and IQRs are in the CSV file. The IQR is often large: for $n \geq 200$ it reaches up to 67% of the median in some configurations, so small differences between methods are not reliable. The largest size tested was $n = 1000$; there were no timeouts. For $n \leq 50$ the times are around one microsecond or less, so they are at the limit of what the timer can measure.

Memory. The peak memory was the same for all methods at each n: it grows from 3.8 MiB ($n = 10$) to 26.5 MiB ($n = 1000$), which is about the three matrices ($3 \cdot 8n^2$ bytes = 22.9 MiB at $n = 1000$) plus about 3.7 MiB of process overhead. The optimisations change the time but not the memory.

Table 1: Part A: median time (ms) of 5 repetitions. Last column: baseline divided by the best blocked version.

n	IJK	IKJ	B=16	B=32	B=64	B=128	speed-up
10	0.0014	0.0009	**0.0010**	0.0013	0.0021	0.0013	1.4
50	0.2611	0.1095	0.1690	0.1033	**0.0329**	0.0374	7.9
100	1.7	1.1	0.9718	**0.7784**	0.8212	1.4	2.2
200	11.2	4.4	**2.8**	6.8	7.7	6.2	4.0
300	33.6	23.0	20.5	17.6	12.4	**12.2**	2.8
400	87.4	23.0	31.7	**19.3**	41.0	34.5	4.5
500	172	57.9	55.9	50.3	47.4	**46.0**	3.7
600	339	81.7	75.5	62.2	**60.0**	111	5.6
800	883	225	203	**153**	174	191	5.8
1000	1914	615	449	410	**384**	410	5.0

4.2 Part B: sparse vs dense

Table 2 compares both methods. $nnz(A) = nnz(B) = \text{round}(d\,n^2)$. The ratio is dense time divided by sparse time, so a value above 1 means that CSR was faster. The last column is the theoretical $nnz(C)$ from Equation 1.

Table 2: Part B: median time in ms (IQR in brackets) of 5 repetitions. Conversion costs are not included.

n	d	$nnz(A)$	Dense	Sparse CSR	Dense/Sparse	$nnz(C)$	theory
100	1%	100	0.11 (0.00)	0.14 (0.07)	0.75	97	100
100	5%	500	0.11 (0.03)	0.36 (0.17)	0.30	2136	2214
100	10%	1000	0.11 (0.00)	0.64 (0.11)	0.17	6343	6340
100	20%	2000	0.11 (0.00)	1.02 (0.14)	0.11	9841	9831
500	1%	2500	9.98 (2.44)	0.75 (0.11)	13.4	12148	12193
500	5%	12500	5.43 (5.74)	8.18 (1.45)	0.66	178650	178486
500	10%	25000	6.64 (36.06)	15.66 (12.06)	0.42	248294	248357
500	20%	50000	4.19 (0.24)	27.21 (10.90)	0.15	250000	250000
1000	1%	10000	27.58 (14.54)	3.54 (0.73)	7.8	95595	95167
1000	5%	50000	23.64 (4.58)	38.92 (27.00)	0.61	917934	918172
1000	10%	100000	27.98 (5.41)	105.68 (25.57)	0.26	999967	999957
1000	20%	200000	42.77 (7.61)	423.00 (23.99)	0.10	1000000	1000000

5 Discussion and Limitations

5.1 Part A

Both optimisations were faster than the baseline in almost all sizes. Loop reordering gave between 1.5 and 4 times less time, which fits the idea that reading B by rows is better than by columns. The best blocked version was 3 to 6 times faster than the baseline for $n \geq 200$, and for $n \geq 500$ it was also faster than loop reordering (for example 384 ms vs. 615 ms at $n = 1000$). However, blocking does not always win: with $B = 64$ it was slower than I-K-J at $n = 200$ (7.7 vs. 4.4 ms) and at $n = 400$ (41.0 vs. 23.0 ms). The best B also changes with n (16, 32, 64 or 128), and at large n the differences between block sizes (for example 384–449 ms at $n = 1000$) are comparable to the IQR, so I cannot say there is one best block size; $B = 32$–64 looked like a reasonable choice. My explanation (better use of L1/L2 cache) is a hypothesis, since I did not measure cache misses.

5.2 Part B

At $n = 100$ the dense method was faster for all densities, because the matrices are so small that the sparse overhead is larger than the savings. For $n \geq 500$ CSR was faster only at 1% density (about 13 times at $n = 500$ and 8 times at $n = 1000$). At 5% density, dense was already faster (0.66 and 0.61 times), so in this experiment the crossover is somewhere between 1% and 5% for $n \geq 500$; I did not test densities in between. At 20% density and $n = 1000$, CSR was 10 times slower. One reason is fill-in: the measured $nnz(C)$ agrees with Equation 1 (within 0.5% for $n \geq 500$) and shows that already at 5% the result has 92% of its entries nonzero for $n = 1000$, so a sparse result has no advantage. Also, CSR needs an index per value and irregular memory access, while BLAS is highly optimised for dense data.

5.3 Limitations

- The experiments ran on a single Windows machine, not a dedicated benchmark server, with only 5 repetitions. The IQRs are often large, so results are approximate.
- The dense times in Part B should not depend on the density (the dense matrices have the same size), but they vary by up to a factor of 2 (for example 23.6–42.8 ms at $n = 1000$), which shows the noise level of those measurements. In Part B I did not limit the number of BLAS threads, so `np.dot` may have used several threads, which favours the dense method.
- In Part B the sparse method is SciPy's, not my own implementation, and its conversion costs are not measured. Only the median and IQR were saved, not every repetition. The memory in Part B was not measured in a way that allows a comparison.
- Only uniformly random sparse matrices were tested; structured matrices (diagonal, banded) could behave very differently. Sizes are small ($n \leq 1000$) and only one machine and compiler setup was used.
- Part A has no separate warm-up and the repetitions run in the same process, so the first repetition is sometimes slower.

6 Conclusions

In my tests, loop reordering and cache blocking made the dense C multiplication up to about 5 times faster than the basic version at $n = 1000$, without using more memory; blocking was the best option for $n \geq 500$, but not at every size and with no clear best block size. For sparse matrices, CSR was only better than dense multiplication at 1% density for $n \geq 500$, and at 5% or more the product is almost dense and the dense method won. These conclusions only apply to the tested sizes, densities and setup. As future work, I would measure cache misses, test densities between 1% and 5%, include conversion costs, and run my own CSR kernel.

Materials and AI Use

Code and data: `https://github.com/Saradevora/Assignment-2` (C kernels, generation, benchmark and validation scripts, and the CSV results).

AI use: I used ChatGPT and Claude to help structure the benchmark scripts, to review my code and results, and to improve the English of this report. I ran the experiments and checked the results myself.

References

[1] R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, Pearson.

[2] M. S. Lam, E. E. Rothberg and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," *ASPLOS IV*, 1991.

[3] F. G. Gustavson, "Two fast algorithms for sparse matrices: multiplication and permuted transposition," *ACM Transactions on Mathematical Software* 4(3), 1978.

[4] P. Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods* 17, 2020.

[5] C. R. Harris et al., "Array programming with NumPy," *Nature* 585, 2020.